

Nama : Hilman Aniq Almunawa

NIM : 109082500027

Kelas : IF-13-05

JAWABAN ASESMEN 3 SEARCHING & SORTING

1. Source Code jawaban :

```
package main

import "fmt"

const NMAX = 1000000

type arrInt [NMAX]int

func sorting(T *arrInt, n int) {
    var step, minIdx, j, sementara int
    for step = 0; step < n-1; step++ {
        minIdx = step
        for j = step + 1; j < n; j++ {
            if T[minIdx] > T[j] {
                minIdx = j
            }
        }
        sementara = T[minIdx]
        T[minIdx] = T[step]
        T[step] = sementara
    }
}

func median(T arrInt, n int) float64 {
    var nilaiTengah int = n / 2
    if n%2 != 0 {
        return float64(T[nilaiTengah])
    }
    return float64(T[nilaiTengah-1]+T[nilaiTengah]) / 2.0
}

func main() {
```

```

var A arrInt
var bilangan int
var jumlah int = 0

fmt.Println("Input data masukan : ")

for {
    _, err := fmt.Scan(&bilangan)

    if err != nil || bilangan == -5313541 {
        break
    }

    if bilangan == 0 {
        sorting(&A, jumlah)
        fmt.Println("Median : ")
        fmt.Println(median(A, jumlah))
    } else {
        if jumlah < NMAX {
            A[jumlah] = bilangan
            jumlah++
        }
    }
}
}

```

Screenshot Output :

```

PS C:\Users\micro\Documents\codingan hilman\github> go run "c:\Users\micro\Documents\codingan hilman\github\109082500027_1.go"
Input data masukan :
7 23 11 0 5 19 2 29 3 13 17 0 -5313541
Median :
11
Median :
12
PS C:\Users\micro\Documents\codingan hilman\github> go run "c:\Users\micro\Documents\codingan hilman\github\109082500027_1.go"
Input data masukan :
23 12 24 26 20 10 25 8 0 -5313541
Median :
21.5
PS C:\Users\micro\Documents\codingan hilman\github> go run "c:\Users\micro\Documents\codingan hilman\github\109082500027_1.go"
Input data masukan :
19 21 34 67 32 1 0 45 32 44 76 0 99 89 0 -5313541
Median :
26.5
Median :
33
Median :
39

```

Penjelasan Singkat Cara Kerja Program :

Program di atas berfungsi untuk mencari dan mencetak nilai median dari sekumpulan data bilangan bulat setiap kali mendeteksi angka 0. Cara kerjanya, program pertama kali membaca input bilangan secara berulang. Lalu program akan melooping dan memasukkan angka ke dalam array selama inputnya bukan angka penanda -5313541. Ketika program membaca angka 0, saya memanggil fungsi sorting. Fungsi ini bekerja menggunakan metode Selection Sort dengan cara mencari angka terkecil dari deretan data yang belum terurut, lalu menukarnya ke posisi paling depan. Proses ini diulang sampai semua angka terurut dari yang terkecil ke terbesar. Setelah array rapi, program memanggil fungsi median untuk menghitung nilai tengahnya. Terakhir, nilai median tersebut dicetak ke layar dan program lanjut membaca sisa angka berikutnya.

2. Source Code Jawaban :

```
package main

import "fmt"

type pemain struct {
    name      string
    gol, assist int
}

const NMAX = 1000

type arrPemain [NMAX]pemain

func main() {
    var dataPemain arrPemain
    var i, j, maxIdx, banyakPemain int
    var namaDepan, namaBelakang string
    var temp pemain

    fmt.Println("Masukkan Data Input : ")

    fmt.Scan(&banyakPemain)
    for i = 0; i < banyakPemain && i < NMAX; i++ {
        fmt.Scan(&namaDepan, &namaBelakang, &dataPemain[i].gol,
            &dataPemain[i].assist)
```

```

        dataPemain[i].name = namaDepan + " " + namaBelakang
    }

    for i = 0; i < banyakPemain-1; i++ {
        maxIdx = i
        for j = i + 1; j < banyakPemain; j++ {
            if dataPemain[j].gol > dataPemain[maxIdx].gol ||
            (dataPemain[j].gol == dataPemain[maxIdx].gol && dataPemain[j].assist >
            dataPemain[maxIdx].assist) {
                maxIdx = j
            }
        }
        temp = dataPemain[maxIdx]
        dataPemain[maxIdx] = dataPemain[i]
        dataPemain[i] = temp
    }

    fmt.Println("\nHasil Sorting : ")
    for i = 0; i < banyakPemain; i++ {
        fmt.Println(dataPemain[i].name, dataPemain[i].gol,
        dataPemain[i].assist)
    }
}

```

Screenshot Output :

```

PS C:\Users\micro\Documents\codingan hilman\github>
2 .go"
Masukkan Data Input :
9
Bruno Fernandes 7 3
Jamie Vardy 8 1
Dominic Calvert-Lewin 10 2
Son Heung-Min 9 2
Harry Kane 7 2
Ollie Watkins 6 1
Patrick Bamford 7 1
Callum Wilson 7 0
Mohamed Salah 8 2

Hasil Sorting :
Dominic Calvert-Lewin 10 2
Son Heung-Min 9 2
Mohamed Salah 8 2
Jamie Vardy 8 1
Bruno Fernandes 7 3
Harry Kane 7 2
Patrick Bamford 7 1
Callum Wilson 7 0
Ollie Watkins 6 1

```

```

PS C:\Users\micro\Documents\codingan hilman>
2 .go"
Masukkan Data Input :
7
Bruno Fernandes 7 3
Dominic Calvert-Lewin 7 12
Son Heung-Min 7 2
Harry Kane 5 2
Patrick Bamford 5 1
Callum Wilson 5 0
Mohamed Salah 3 2

Hasil Sorting :
Dominic Calvert-Lewin 7 12
Bruno Fernandes 7 3
Son Heung-Min 7 2
Harry Kane 5 2
Patrick Bamford 5 1
Callum Wilson 5 0
Mohamed Salah 3 2

```

Penjelasan Singkat Cara Kerja Program :

Program di atas berfungsi untuk mengurutkan peringkat pemain sepak bola dari yang terbesar ke terkecil berdasarkan jumlah gol dan assist. Cara kerjanya, program pertama kali membaca jumlah pemain (n). Lalu program akan melooping sebanyak n kali untuk membaca nama, jumlah gol, dan jumlah assist ke dalam sebuah array (struct). Setelah data masuk, saya melakukan proses pengurutan menggunakan metode Selection Sort. Program bekerja dengan cara mencari pemain dengan jumlah gol terbanyak, lalu menukarnya ke posisi paling atas. Kalau ada pemain yang jumlah golnya sama, program akan mengecek jumlah assist-nya untuk menentukan siapa yang ditaruh di atas. Proses ini diulang sampai semua pemain terurut dengan benar. Terakhir, daftar klasemen yang sudah rapi dicetak berderet ke bawah.

3. Source Code Jawaban :

```
package main

import "fmt"

const NMAX = 1000000

type partai struct {
    nama, suara int
}

type tabPartai [NMAX]partai

func main() {
    var p tabPartai
    var n, inputNomor, posisiPartai int

    n = 0

    fmt.Println("Masukkan proses input suara : ")

    fmt.Scan(&inputNomor)
    for inputNomor != -1 {
        posisiPartai = posisi(p, n, inputNomor)
        if posisiPartai == -1 {
            p[n].nama = inputNomor
            p[n].suara = 1
            n++
        } else {
```

```

        p[posisiPartai].suara++
    }
    fmt.Scan(&inputNomor)
}

var i, j int
var temp partai
for i = 1; i < n; i++ {
    temp = p[i]
    j = i
    for j > 0 && p[j-1].suara < temp.suara {
        p[j] = p[j-1]
        j--
    }
    p[j] = temp
}

fmt.Println("\nHasil Perhitungan suara : ")
for i = 0; i < n; i++ {
    fmt.Printf("%v(%v) ", p[i].nama, p[i].suara)
}
fmt.Println()
}

func posisi(t tabPartai, n int, nama int) int {
    var idx int
    for idx = 0; idx < n; idx++ {
        if t[idx].nama == nama {
            return idx
        }
    }
    return -1
}

```

Screenshot Output :

```
PS C:\Users\micro\Documents\codingan hilman\github> go run "c:\Users\micro\Documents\codingan hilman\github\10908250027_Hilman-Aniq-Almunawa\Asesment 3\Soal 3 Asesment 3\Soal 3 .go"
Masukkan proses input suara :
5 8 8 6 6 6 8 6 7 5 6 8 7 7 6 6 6 5 8 7 6 7 8 8 7 8 7 5 8 7 8 5 7 6 6 6 5 6 8 6 6 7 7 8 7 5 6 8 8 5
6 6 5 5 7 8 5 8 7 7 8 5 7 7 5 6 7 6 5 8 8 7 6 7 8 7 6 8 7 5 6 6 5 8 8 8 6 6 6 8 6 7 8 7 8 5 8 -1

Hasil Perhitungan suara :
8(29) 6(28) 7(24) 5(17)
PS C:\Users\micro\Documents\codingan hilman\github> go run "c:\Users\micro\Documents\codingan hilman\github\10908250027_Hilman-Aniq-Almunawa\Asesment 3\Soal 3 Asesment 3\Soal 3 .go"
Masukkan proses input suara :
14 10 13 13 14 10 11 13 13 12 15 11 10 -1

Hasil Perhitungan suara :
13(4) 10(3) 14(2) 11(2) 12(1) 15(1)
PS C:\Users\micro\Documents\codingan hilman\github> go run "c:\Users\micro\Documents\codingan hilman\github\10908250027_Hilman-Aniq-Almunawa\Asesment 3\Soal 3 Asesment 3\Soal 3 .go"
Masukkan proses input suara :
-1

Hasil Perhitungan suara :
```

Penjelasan Singkat Cara Kerja Program :

Program di atas berfungsi untuk merekap dan mengurutkan hasil perolehan suara partai dari yang terbesar ke terkecil. Cara kerjanya, program membaca input nomor partai secara berulang sampai mendeteksi angka -1. Setiap ada nomor yang masuk, saya memanggil fungsi posisi yang menggunakan metode pencarian sekuensial (Sequential Search) untuk mengecek apakah partai tersebut sudah terdaftar di array. Kalau belum ada, data partai baru dimasukkan dengan perolehan 1 suara, tapi kalau sudah ada, jumlah suaranya tinggal ditambah satu. Setelah semua data masuk, saya melakukan pengurutan menggunakan metode Insertion Sort. Fungsi ini bekerja dengan cara menyisipkan partai yang suaranya lebih besar ke posisi depan dan menggeser data yang suaranya lebih kecil ke belakang. Terakhir, array yang sudah rapi dicetak berderet ke samping.